

Recherche de documents « Document Retrieval »

Séance 6

Embeddings, visualisation lexicale et gestion des descripteurs

I. Introduction :

Cette séance s'inscrit dans la continuité directe du projet de recherche documentaire et marque une étape clé dans l'évolution du moteur de recherche développé jusqu'ici. Après avoir mis en œuvre des méthodes classiques de représentation des textes fondées sur des descripteurs symboliques, cette séance introduit une nouvelle famille de descripteurs fondée sur les plongements vectoriels (embeddings).

Les embeddings offrent des représentations plus compactes et plus expressives des contenus textuels, mieux adaptées aux approches contemporaines de la recherche d'information. Ils permettent notamment de modéliser des relations de similarité sémantique plus fines entre mots, phrases et documents, au-delà de la simple cooccurrence lexicale. Les différences fondamentales entre descripteurs symboliques et embeddings seront rappelées et formalisées dans la section suivante.

Contrairement aux séances précédentes, l'objectif n'est plus de réimplémenter les modèles sous-jacents, mais de les intégrer et de les exploiter concrètement au sein d'un moteur de recherche opérationnel. La séance met ainsi l'accent sur la manipulation de modèles existants, leur utilisation comme descripteurs, et leur articulation avec les mécanismes de recherche déjà étudiés.

En complément des embeddings, la séance introduit également :

- des outils de visualisation lexicale destinés à l'analyse exploratoire et à l'interprétation des contenus et des résultats ;
- une gestion structurée des descripteurs (calcul, stockage, rechargement, indexation), indispensable pour passer à une interface interactive lors de la séance 7.

II. Plan de la sixième séance étape de l'atelier

1. Rappels : descripteurs symboliques vs plongements « embeddings »

Les séances précédentes ont permis de manipuler différentes méthodes de vectorisation de documents reposant sur des statistiques lexicales. Ces approches constituent le socle historique de la recherche documentaire et sont indispensables pour comprendre les mécanismes d'indexation et de calcul de similarité.

Avant d'introduire et d'exploiter les plongements vectoriels, cette section propose un rappel structuré des descripteurs symboliques, puis présente les embeddings comme une famille

de représentations complémentaires. L'objectif est de mettre en évidence les différences de nature, de représentation et d'usage entre ces deux familles de descripteurs.

a. Les descripteurs symboliques

Les descripteurs symboliques représentent les documents à partir d'unités lexicales explicites, (mots, n-grammes), issues du vocabulaire du corpus. Des méthodes comme Bag-of-Words, TF, TF-IDF ou BM25 produisent des vecteurs de grande dimension, où chaque composante correspond à une unité linguistique identifiée.

Ces représentations reposent exclusivement sur des statistiques de fréquence et conduisent généralement à des vecteurs très clairsemés. Elles présentent l'avantage d'être simples à implémenter et facilement interprétables, mais elles souffrent de plusieurs limitations importantes : une forte dépendance au vocabulaire exact, l'absence de prise en compte explicite de la similarité sémantique, et une sensibilité marquée aux variations lexicales. Ces limitations apparaissent notamment lorsque la requête et les documents pertinents utilisent des formulations différentes. Elles motivent l'introduction de représentations complémentaires, capables de capturer des relations plus riches entre les unités linguistiques.

b. Les plongements « embeddings »

Les plongements vectoriels constituent une alternative aux descripteurs symboliques. Ils associent à chaque unité linguistique (mot, phrase, document) un vecteur dense de dimension fixe relativement faible, appris à partir de données textuelles. Les plongements projettent les unités linguistiques dans un espace continu, où la proximité géométrique entre vecteurs reflète une similarité sémantique généralement mesurée par la similarité cosinus. Ces représentations sont aujourd'hui largement utilisées en recherche d'information et en traitement automatique des langues, car elles permettent de dépasser la simple correspondance lexicale et de capturer des régularités sémantiques à différents niveaux. Dans cette séance, les embeddings sont utilisés comme descripteurs vectoriels du moteur de recherche, en complément des représentations symboliques, afin d'enrichir les mécanismes de comparaison entre requêtes et documents.

2. Niveaux de plongement et granularité des représentations

Les plongements vectoriels peuvent être construits à différents niveaux de granularité selon l'unité linguistique que l'on souhaite représenter dans le moteur de recherche. Selon le niveau choisi (mot, phrase, document), les méthodes de calcul, d'agrégation et d'utilisation des vecteurs diffèrent. Cette section présente les principaux niveaux de plongement utilisés dans cette séance :

- **Plongement de mots**, la brique de base des représentations vectorielles ;
- **Plongement de phrases**, obtenu par composition ou agrégation des vecteurs de mots ;
- **Plongement de documents**, obtenu soit par agrégation des vecteurs de mots ou de phrases, soit par l'apprentissage d'un modèle dédié au niveau document (Doc2Vec).

L'objectif est de définir un cadre cohérent permettant de vectoriser aussi bien le corpus que les requêtes, et d'exploiter ces représentations de manière uniforme au sein du moteur de recherche.

Dans ce cadre, deux paramètres centraux sont utilisés dans toutes les fonctions de calcul de plongements :

- **strategie_agregation** : méthode d'agrégation des vecteurs (par exemple moyenne, somme ou moyenne pondérée) ;
- **strategie_oov** : gestion des mots hors vocabulaire (out-of-vocabulary). Les valeurs possibles sont 'ignore' (le mot est ignoré et aucun vecteur n'est retourné (None)), 'zero' (vecteur nul de dimension `modele_embeddings.vector_size`), 'fasttext' (vecteur produit à partir des sous-mots si le modèle le permet) ou 'error' (exception levée).

Dans la suite de cette séance, le terme **modèle d'embeddings** désigne un objet retourné par la bibliothèque Gensim (Word2Vec, FastText ou modèle pré-entraîné équivalent), Ce modèle permet notamment :

- d'accéder aux vecteurs associés aux unités linguistique via « `modele_embeddings.wv[token]` » ;
- de tester la présence d'un mot dans le vocabulaire via « `token in modele_embeddings.wv` » ;
- de calculer des similarités et d'extraire des voisins sémantiques.

Ces mécanismes garantissent la cohérence des calculs de plongements pour toutes les granularités et facilitent l'expérimentation avec différentes stratégies de gestion des mots absents. Les fonctions décrites dans les sous-sections suivantes exploitent ces paramètres de manière uniforme, en se basant sur les représentations textuelles déjà construites.

a. Plongement de mots

Le plongement de mots constitue le niveau de granularité fondamental des représentations vectorielles utilisées dans le moteur de recherche. A chaque mot du vocabulaire est associé un vecteur dense appris à partir de données textuelles, capturant des régularités et des proximités sémantiques entre termes. Ces vecteurs servent de base aux représentations de phrases et de documents introduites dans les sections suivantes.

Dans le cadre de cette séance, les plongements de mots sont utilisés à la fois pour la vectorisation du corpus et pour celle des requêtes. Deux approches complémentaires sont proposées : l'apprentissage d'un modèle sur le corpus du projet et l'utilisation de modèles pré-entraînés.

i. Apprentissage d'un modèle de plongement sur le corpus

Il est possible d'apprendre des plongements de mots directement à partir du corpus utilisé dans le projet. Cette approche permet d'obtenir des vecteurs adaptés au vocabulaire et au domaine des textes étudiés, au prix d'une qualité sémantique dépendante de la taille et de la richesse du corpus. Vous utiliserez des modèles de

type `Word2Vec` ou `FastText`, entraînés sur le corpus prétraité (tokens, lemmes ou formes canoniques).

L'apprentissage d'un modèle comprend les étapes suivantes :

- construction du vocabulaire à partir du corpus ;
- apprentissage des vecteurs de mots ;
- accès aux vecteurs associés à chaque mot du vocabulaire.

Les paramètres du modèle (dimension des vecteurs, fenêtre de contexte, fréquence minimale, nombre d'itérations) influencent directement la qualité et la structure de l'espace vectoriel.

- Écrire une fonction **`entraîner_model_word2vec(corpus, taille_vecteur, fenetre, min_count, epochs)`** qui entraîne un modèle `Word2Vec` sur le corpus et retourne un objet `modele_embeddings`, prêt à fournir les vecteurs des mots.
- Écrire une fonction équivalente **`entraîner_model_fasttext(corpus, taille_vecteur, fenetre, min_count, epochs)`** qui entraîne un modèle `FastText` sur le corpus et retourne un objet `modele_embeddings`, capable de produire des vecteurs pour certains mots hors vocabulaire grâce aux sous-mots.

ii. Utilisation de modèles de plongements pré-entraînés

Une alternative à l'apprentissage local consiste à utiliser des modèles de plongements pré-entraînés sur de très grands corpus.

Ces modèles offrent :

- une couverture lexicale plus large ;
- des représentations sémantiques plus stables ;
- une meilleure robustesse aux variations lexicales.

Dans le cadre de ce projet, vous pouvez utiliser :

- des modèles `Word2Vec` pré-entraînés ;
- des modèles `FastText` pré-entraînés, qui permettent de produire des vecteurs pour des mots hors vocabulaire grâce à l'utilisation de sous-mots.

Lors de l'utilisation d'un modèle pré-entraîné, seuls les mots présents dans le vocabulaire du modèle (ou reconstruits par sous-mots pour `FastText`) peuvent être vectorisés.

- Écrire une fonction **`charger_model_preentraîne(chemin, type_model='word2vec')`** qui charge un modèle pré-entraîné et retourne un objet `modele_embeddings`. Le paramètre `type_model` permet d'indiquer la famille du modèle de plongements à charger (`'word2vec'`, `'fasttext'`), les autres caractéristiques du modèle (dimension des vecteurs, vocabulaire, paramètres d'entraînement, format de stockage) sont propres au modèle chargé. Elles sont automatiquement déterminées lors du chargement
- Écrire une fonction **`get_vecteur_mot(token, modele_embeddings, strategie_oov='ignore')`** qui retourne le vecteur associé à un mot à partir du

modèle `modele_embeddings` sinon le vecteur retournée dépendra de la valeur du paramètre `strategie_oov`.

b. Plongement de phrases

Le plongement de phrases vise à représenter une séquence de mots par un vecteur unique, afin de permettre la comparaison sémantique entre unités textuelles plus longues que le mot isolé. Une phrase peut être considérée comme un court document, et il existe des modèles capables d'apprendre directement des représentations vectorielles à ce niveau. Dans le cadre de cette séance, toutefois, les plongements de phrases sont construits à partir des plongements de mots, par des méthodes de composition simples. Ce choix permet de s'appuyer sur les représentations lexicales introduites précédemment, tout en conservant une mise en œuvre légère et directement contrôlable.

Plusieurs méthodes simples d'agrégation peuvent être utilisées pour calculer le vecteur d'une phrase à partir des vecteurs de ses mots :

- moyenne : la moyenne arithmétique des vecteurs de mots ;
- somme : la somme des vecteurs de mots ;
- pondérée : la moyenne pondérée des vecteurs, par exemple à l'aide de poids TF ou TF-IDF.

Les mots absents du vocabulaire sont traités selon la stratégie `strategie_oov` définie dans la section 2.

- Écrire une fonction `plongement_phrase_par_mots(phrase, modele_embeddings, strategie_agregation='moyenne', strategie_oov='ignore')` qui retourne le vecteur de la phrase en agrégeant les vecteurs de mots selon la stratégie choisie.
- Écrire une fonction `similarite_phrases(phrase1, phrase2, modele_embeddings, strategie_agregation='moyenne', strategie_oov='ignore', mesure='cosinus')` qui calcule la similarité entre deux phrases en utilisant les vecteurs agrégés.
- Écrire une fonction `top_k_phrases_similaires(phrase, corpus_phrases, modele_embeddings, strategie_agregation='moyenne', strategie_oov='ignore', k=5)` qui retourne les k phrases les plus proches sémantiquement dans un corpus donné.

c. Plongement de documents

Le plongement de documents permet de représenter un texte complet par un vecteur unique, afin de permettre la comparaison sémantique entre documents ou entre une requête et un document du corpus. Ce niveau de représentation est central dans le moteur de recherche, puisqu'il conditionne directement le calcul des scores de similarité et le classement des résultats.

Dans cette séance, deux approches complémentaires sont proposées pour obtenir des vecteurs de documents :

- l'agrégation de vecteurs calculés à des niveaux inférieurs (mots ou phrases) ;
- l'apprentissage d'un modèle dédié au niveau document, tel que *Doc2Vec*.

i. Agrégation des vecteurs de mots ou de phrases

Une première approche consiste à construire le vecteur d'un document par agrégation des vecteurs associés à ses unités constitutives. Selon le niveau choisi, cette agrégation peut s'effectuer :

- directement à partir des vecteurs de mots du document ;
- à partir des vecteurs de phrases obtenus comme décrit en section II.2.b.

Les stratégies d'agrégation usuelles incluent notamment :

- la **moyenne** des vecteurs ;
- la **somme** des vecteurs ;
- une **agrégation pondérée**, par exemple à l'aide de poids TF ou TF-IDF.

Les mots hors vocabulaire sont pris en charge via le paramètre **strategie_oov**, conformément aux règles définies dans la section 2.

- Écrire une fonction **plongement_document_par_mots(document, modele_embeddings, strategie_agregation='moyenne', strategie_oov='ignore')** qui retourne un vecteur représentant le document à partir de l'agrégation des vecteurs de mots.
- Écrire une fonction **plongement_document_par_phrases(document, modele_embeddings, strategie_agregation_phrase='moyenne', strategie_agregation_doc='moyenne', strategie_oov='ignore')** qui retourne un vecteur représentant le document à partir de l'agrégation des vecteurs de phrases.

Cette approche présente l'avantage de la simplicité et de la réutilisation directe des modèles de plongement de mots, mais elle repose sur des hypothèses fortes d'additivité sémantique.

ii. Apprentissage d'un modèle de plongement de documents (Doc2Vec)

Une alternative consiste à apprendre directement des vecteurs de documents à l'aide d'un modèle dédié, tel que *Doc2Vec*. Contrairement aux méthodes par agrégation, ce type de modèle apprend une représentation spécifique pour chaque document, optimisée pour prédire les contextes lexicaux observés dans le corpus.

L'apprentissage d'un modèle *Doc2Vec* comprend notamment :

- l'association d'un identifiant unique à chaque document ;
 - l'apprentissage conjoint des vecteurs de documents et de mots ;
 - l'inférence de vecteurs pour de nouveaux documents ou requêtes.
-
- Écrire une fonction **entraîner_model_doc2vec(corpus, taille_vecteur, fenetre, min_count, epochs)** qui entraîne un modèle Doc2Vec sur le corpus fourni et retourne un objet Gensim **modele_doc2vec**.
 - Écrire une fonction **plongement_document_doc2vec(document, modele_doc2vec)** qui infère le vecteur associé à un document ou à une requête à partir d'un modèle Doc2Vec entraîné.

iii. Utilisation des vecteurs de documents pour la recherche

Une fois les vecteurs de documents calculés, cette représentation unifiée permet :

- de comparer des documents entre eux ;
 - de comparer des requêtes et des documents ;
 - de construire un classement sémantique des résultats de recherche.
-
- Écrire une fonction **similarite_documents(doc1, doc2, modele_embeddings, mesure='cosinus')** qui calcule la similarité entre deux documents. Elle suppose que **modele_embeddings** permet d'obtenir une représentation vectorielle des documents, soit en fournissant directement des vecteurs de documents déjà calculés, soit en permettant de calculer ces vecteurs à la volée à partir d'un modèle de plongements (Word2Vec, FastText ou Doc2Vec), selon les méthodes définies précédemment. La similarité est ensuite calculée à partir des vecteurs obtenus, selon la mesure spécifiée.
 - Écrire une fonction **top_k_documents_similaires(document, corpus, modele_embeddings, k=5)** qui retourne les documents les plus proches sémantiquement.

3. Visualisation lexicale et interprétation des contenus

Les méthodes de recherche documentaire reposent sur des représentations vectorielles, qu'elles soient symboliques ou fondées sur des plongements, et sur des calculs de similarité qui sont par nature abstraits et difficilement interprétables par un utilisateur humain. Or, dans un contexte de recherche documentaire, il est essentiel de pouvoir analyser, expliquer et justifier les résultats produits par le moteur de recherche, tant pour le concepteur du système que pour l'utilisateur final. Afin de faciliter l'analyse exploratoire d'un corpus et la compréhension des résultats retournés, il est donc utile de compléter les représentations numériques par des outils de visualisation lexicale.

Cette section introduit les nuages de mots (word clouds) comme un moyen simple et intuitif de représenter visuellement le contenu lexical d'un document, d'un sous-corpus ou d'un ensemble de résultats de recherche. Ces visualisations jouent un rôle complémentaire aux embeddings : elles ne servent pas au calcul de similarité, mais à l'exploration, à l'interprétation et à la présentation des résultats, en établissant un lien entre les calculs numériques du moteur de recherche et une représentation plus accessible des informations manipulées. Les nuages de mots sont ainsi utilisés comme un support d'analyse permettant d'explorer un corpus ou un document, de comprendre quels termes contribuent à une représentation ou à un score de similarité, et de préparer l'interprétation des mécanismes d'expansion de requête introduits dans la section suivante, notamment dans la perspective de l'interface interactive de la séance 7.

a. Les nuages de mots comme outil d'analyse exploratoire

Les nuages de mots constituent un outil simple et efficace pour visualiser la distribution des termes dans un texte ou un ensemble de documents. Chaque mot est affiché avec une taille proportionnelle à une valeur associée, généralement une fréquence ou un poids.

Dans le cadre du moteur de recherche, les nuages de mots peuvent être utilisés pour :

- explorer rapidement le contenu d'un document ou d'un corpus en visualisant les termes (ou thèmes) dominants ;
- comparer visuellement plusieurs documents ou sous-corpus ;
- analyser l'impact des étapes de prétraitement (suppression des stopwords, lemmatisation, etc.) ;
- visualiser des documents retournés par une requête.

Les valeurs utilisées pour définir l'importance des mots et construire un nuage de mots peuvent provenir de différentes sources :

- fréquence brute des mots dans un document ou un corpus ;
- scores pondérés (TF, TF-IDF, BM25, etc.) ;
- scores calculés à partir de résultats de recherche ou de similarité.

Dans le cadre de ce projet, les nuages de mots sont utilisés comme outil d'analyse et de restitution, et non comme descripteurs du moteur de recherche.

b. Création de nuages de mots à partir d'un texte

Dans un premier temps, un nuage de mots peut être généré directement à partir du texte brut d'un document ou d'un corpus. Cette approche repose sur les fréquences des mots, éventuellement après suppression des mots vides (stopwords). Les paramètres de génération permettent de contrôler l'apparence du nuage (dimensions, couleurs, arrière-plan, masque, etc.).

- Écrire une fonction **generer_nuage_mots_texte(texte, stopwords=None, largeur=600, hauteur=300, couleur_fond='white')** qui génère et affiche un nuage de mots à partir d'un texte donné.
- Écrire une fonction **nuage_mots_document(document, pretraitement=True)** qui applique éventuellement un prétraitement (tokenisation, suppression des stopwords) avant de générer le nuage de mots.
- Écrire une fonction **nuage_mots_sous_corpus(liste_documents)** qui concatène les textes d'un sous-corpus et produit un nuage de mots global.

c. Nuages de mots pondérés par des scores (TF, TF-IDF, etc.)

Au-delà des fréquences brutes, il est possible de générer des nuages de mots à partir de dictionnaires associant chaque mot à un score numérique. Cette approche permet de visualiser non pas les mots les plus fréquents, mais les mots les plus discriminants ou les plus représentatifs selon un critère donné (par exemple TF-IDF).

Cette méthode est particulièrement intéressante pour :

- mettre en évidence les mots spécifiques à un document ;
- comparer des documents similaires ;
- interpréter les résultats d'une recherche documentaire.

- Écrire une fonction **nuage_mots_pondere(poids_par_mot)** qui génère un nuage de mots à partir d'un dictionnaire générique {mot: poids} indépendamment de l'origine des pondérations.
- Écrire une fonction **comparer_nuages_documents(doc1, doc2, scores1, scores2)** permettant d'afficher côte à côte deux nuages de mots pour comparer visuellement deux documents.

d. Visualisation des résultats de recherche par des nuages de mots

Les nuages de mots peuvent également être utilisés pour interpréter les résultats retournés par le moteur de recherche. Lorsqu'une requête est soumise au moteur de recherche, celui-ci calcule des scores de similarité entre la requête et les documents du corpus à partir de descripteurs vectoriels, dont l'interprétation reste toutefois difficile sans support visuel.

Par exemple, après une requête, il est possible de :

- générer un nuage de mots pour chaque document retourné ;
- produire un nuage global à partir des k documents les plus similaires ;
- pondérer les mots par les scores de similarité ou par des scores TF-IDF locaux.

Cette approche permet de relier les résultats numériques du moteur de recherche à une représentation visuelle compréhensible par l'utilisateur.

- Écrire une fonction **nuage_document_resultat(document, poids)** qui affiche un nuage de mots pour un document retourné par le moteur, en utilisant les pondérations associées.
- Écrire une fonction **nuage_mots_resultats_recherche(documents, scores=None)** qui génère un nuage de mots à partir des documents retournés par une requête.
- Écrire une fonction **nuage_mots_top_k_documents(requete, corpus, modele_embeddings, k=5)** qui affiche un nuage de mots représentant les documents les plus proches sémantiquement d'une requête.
- Écrire une fonction **nuage_requete_vs_document(requete, document, poids)** qui permet de visualiser conjointement les termes de la requête et ceux du document, afin d'analyser leur recouvrement lexical.

4. Expansion de requête à l'aide des plongements vectoriels

Les visualisations lexicales introduites dans la section précédente ont permis d'analyser et d'interpréter les contenus textuels ainsi que les résultats retournés par le moteur de recherche. Elles ont également mis en évidence une limite fondamentale des approches strictement lexicales : la dépendance à la forme exacte des mots utilisés dans la requête.

Dans de nombreux cas, une requête courte ou formulée avec un vocabulaire différent de celui des documents pertinents peut conduire à des résultats incomplets, voire à l'absence de résultats. Afin de dépasser cette limitation, il est possible d'enrichir automatiquement la requête en y intégrant des termes sémantiquement proches. Ce principe est connu sous le nom d'expansion de requête.

Cette section introduit l'expansion de requête comme un mécanisme central reliant :

- les plongements vectoriels,
- le calcul de similarité sémantique,
- et le fonctionnement global du moteur de recherche.

a. Principe de l'expansion de requête

L'expansion de requête consiste à enrichir une requête initiale en ajoutant de nouveaux termes jugés pertinents, afin d'améliorer la couverture sémantique de la recherche. Contrairement à une reformulation manuelle, l'expansion est ici réalisée automatiquement à partir des représentations vectorielles.

À partir d'un terme de la requête, les plongements permettent d'identifier d'autres termes proches dans l'espace sémantique, c'est-à-dire des mots dont les vecteurs sont proches selon une mesure de similarité (généralement la similarité cosinus). Ces termes peuvent correspondre à :

- des synonymes ;
- des variantes lexicales ;
- des termes contextuellement proches dans le corpus.

L'expansion de requête vise principalement à :

- améliorer le rappel du moteur de recherche ;
- réduire l'impact des variations lexicales ;
- exploiter la structure sémantique apprise par les embeddings.

Dans la chaîne de traitement du moteur de recherche, l'expansion intervient après l'analyse de la requête et avant la recherche proprement dite :



b. Mise en œuvre de l'expansion de requête

Dans cette séance, l'expansion de requête est mise en œuvre à partir des plongements de mots calculés ou chargés précédemment. Pour chaque terme de la requête, les k mots les plus proches dans l'espace vectoriel sont extraits, puis combinés à la requête initiale.

Plusieurs choix méthodologiques doivent être pris en compte :

- le nombre de termes ajoutés (paramètre k) ;
- la pondération des termes issus de l'expansion ;
- la prise en compte ou non des mots absents du vocabulaire.

L'expansion peut être utilisée de différentes manières :

- pour enrichir une requête lexicale (TF, TF-IDF, BM25) ;

- pour construire une requête vectorielle plus riche ;
- pour guider la recherche de phrases ou de documents similaires.
- Écrire une fonction **termes_expansion(requete, modele_embeddings, k=5)** qui extrait, pour chaque mot de la requête, les k termes les plus proches dans l'espace vectoriel.
- Écrire une fonction **construire_requete_etendue(requete, termes_expanses, alpha=1.0, beta=0.5)** qui combine la requête initiale et les termes d'expansion, en appliquant des pondérations distinctes.
- Écrire une fonction **recherche_avec_expansion(requete, corpus, modele_embeddings, methode)** qui intègre l'expansion dans le moteur de recherche et retourne les documents les plus pertinents.

c. Visualisation de l'espace sémantique pour l'expansion de requête

L'expansion de requête repose sur des relations sémantiques apprises par les plongements vectoriels. Afin de comprendre et d'évaluer la pertinence de cette expansion, il est essentiel de rendre ces relations visibles.

Les visualisations introduites en section 3 peuvent être réutilisées ici pour représenter

- le voisinage sémantique d'un terme de la requête ;
- les termes ajoutés lors de l'expansion ;
- leur importance relative dans la requête étendue.

À partir d'un mot de la requête, on peut construire un nuage de mots représentant ses voisins sémantiques, où la taille de chaque mot est proportionnelle à sa similarité avec le mot de référence. Cette visualisation permet :

- de vérifier qualitativement la cohérence de l'expansion ;
- de comparer différents modèles de plongements ;
- d'illustrer l'impact de l'expansion sur la recherche.
- Écrire une fonction **mots_plus_proches(mot, modele_embeddings, k=20)** qui retourne les k mots les plus proches d'un mot donné, accompagnés de leur score de similarité.
- Écrire une fonction **nuage_expansion_requete(mot, modele_embeddings, k=20)** qui génère un nuage de mots représentant les termes utilisés pour l'expansion.
- Écrire une fonction **comparer_expansion_requete(mot, modele_embeddings1, modele_embeddings2, k=20)** qui permet de comparer visuellement l'expansion produite par deux modèles de plongements différents.

Dans le cadre du moteur de recherche, ces visualisations peuvent être utilisées pour :

- expliquer pourquoi certains documents ont été retrouvés ;
- justifier l'ajout de termes dans la requête étendue ;
- proposer à l'utilisateur des suggestions de reformulation ou d'enrichissement de requête.

5. Gestion des descripteurs et indexation

La gestion efficace des descripteurs constitue un élément central pour rendre le moteur de recherche opérationnel et réactif. Après avoir étudié les différentes familles de descripteurs, symboliques et vectoriels, et leur utilisation à plusieurs niveaux de granularité, cette section aborde les mécanismes permettant d'organiser, de calculer, d'indexer et d'optimiser ces descripteurs. L'objectif est de préparer des structures cohérentes, efficaces et réutilisables, capables de supporter ultérieurement la recherche interactive, sans encore aborder le calcul des scores ni l'interface utilisateur.

Cette section adopte une approche progressive, allant de la structuration initiale des descripteurs (pré-indexation) jusqu'à la mise en place de structures d'indexation symboliques et vectorielles, complétées par des pré-calculs destinés à accélérer les opérations de similarité. Elle met l'accent sur la modularité des représentations, la traçabilité des paramètres de calcul et la réutilisation des descripteurs déjà explorés dans les séances précédentes.

Les objectifs principaux sont les suivants :

- **Centraliser et organiser les descripteurs** : associer à chaque document ou phrase du corpus l'ensemble des descripteurs calculés, en les regroupant dans des structures unifiées et cohérentes.
- **Garantir la compatibilité entre descripteurs** : permettre la coexistence et la manipulation uniforme de représentations symboliques et vectorielles au sein du moteur de recherche.
- **Préparer l'indexation symbolique et vectorielle** : construire des vues dérivées (matrices, mappings, collections de vecteurs) facilitant l'accès aux descripteurs.
- **Optimiser les structures pour la recherche** : effectuer des pré-calculs et organiser les données afin d'accélérer les calculs de similarité et de préparer l'expansion de requête.
- **Faciliter la réutilisation des descripteurs** : permettre la persistance et le rechargement des structures sans recalcul, afin de favoriser l'expérimentation et la comparaison de méthodes.
- **Préparer l'intégration du moteur de recherche dans interface Homme-Machine** : fournir des structures prêtes à être exploitées lors de la séance suivante consacrée à l'interaction utilisateur.

La présente section définit une architecture générale et des principes d'organisation des descripteurs, sans imposer une implémentation unique. Les choix précis de structures de données, de formats de stockage et de stratégies d'indexation relèvent de la conception du système et sont laissés à votre appréciation, à condition de garantir la cohérence, la traçabilité et la réutilisation des descripteurs.

a. Structuration des descripteurs (pré-indexation)

Avant de pouvoir indexer et exploiter efficacement les descripteurs, il est essentiel de structurer les données calculées lors des séances précédentes. Cette phase, que l'on appelle *pré-indexation*, vise à organiser les descripteurs symboliques et vectoriels afin de faciliter leur réutilisation, leur persistance et leur accès rapide lors des calculs de similarité ou de l'expansion de requête.

Les objectifs de la structuration sont :

- **Centraliser les descripteurs** : regrouper pour chaque unité linguistique (phrase ou document) tous les descripteurs calculés, quels que soient leur type ou leur configuration de prétraitement.
- **Assurer la cohérence** : maintenir un lien explicite entre chaque vecteur et les choix méthodologiques (normalisation, pondération, stratégie d'agrégation).
- **Préparer la réutilisation et la persistance** : permettre de sauvegarder et recharger facilement les descripteurs, sans recalcul.
- **Faciliter l'accès aux embeddings** : pour préparer l'expansion de requête et les calculs vectoriels de similarité.

Chaque unité linguistique (phrase ou document) est représentée par une fiche modulaire contenant :

- **Identifiant unique** (id_phrase ou id_document) ;
- **Type d'unité** (phrase ou document) ;
- **Texte original** et/ou les **tokens** issus de la configuration de prétraitement ;
- **Configuration de prétraitement utilisée (config_id)** ;
- **Descripteurs vectoriels** représentant la même unité, calculés selon différentes méthodes (BOW, TF, TF-IDF, BM25, embeddings, etc.) ;
- **Paramètres de calcul du descripteur** :
 - Normalisation appliquée (L1, L2, aucune) pour les descripteurs symboliques ;
 - Stratégie d'agrégation et de pondération (moyenne, TF, TF-IDF, etc.) pour les embeddings ;
- **Métadonnées descriptives**, telles que la longueur du document, le nombre de tokens, la langue ou l'appartenance à un sous-corpus (None ou 0 pour le corpus global), etc ;
- **Statut du descripteur** : calculé ou chargé depuis une sauvegarde sur le disque.

Chaque fiche est conservée dans un dictionnaire Python unique, indexé par l'identifiant de l'unité. Les attributs comme langue, corpus ou configuration de prétraitement sont des propriétés de la fiche, et non des niveaux hiérarchiques imbriqués, afin de simplifier l'accès à la manipulation.

Pour préparer l'expansion de requête et le calcul rapide des vecteurs de phrases ou de documents, il est utile de disposer d'une structure de consultation rapide associant chaque mot à son vecteur. Ce dictionnaire peut être construit à partir du modèle d'embeddings déjà entraîné ou chargé en mémoire. Il permet de :

- Calculer rapidement les vecteurs de phrases ou documents par agrégation de mots.
- Identifier les termes les plus proches dans l'espace vectoriel pour l'expansion de requête.
- Visualiser le voisinage sémantique d'un mot pour l'analyse exploratoire.

Cette organisation facilite la navigation entre différentes représentations, permet la réutilisation des descripteurs et prépare leur persistance et leur indexation pour la recherche vectorielle ou hybride.

- Écrire une fonction **initialiser_fiche_unite(id_unite, type_unite, texte, config)** qui crée une fiche vide, prête à recevoir des descripteurs et métadonnées.
- Écrire une fonction **initialiser_dictionnaire_global()** qui crée le dictionnaire principal regroupant toutes les fiches.
- Écrire une fonction **ajouter_fiche(dico_global, fiche)** qui insère une fiche dans le dictionnaire global en respectant la clé correspondant à l'identifiant de l'unité linguistique.
- Écrire la fonction **filtrer_fiches(dico_global, langue=None, config_id=None, type_unite=None)** qui permet d'extraire un sous-ensemble des fiches selon la langue, la configuration de prétraitement ou le type d'unité linguistique.
- Écrire une fonction **initialiser_vocabulaire_embeddings(modele_embeddings)** qui construit un dictionnaire **mot -> vecteur** pour tous les mots du vocabulaire du modèle, afin de faciliter l'expansion de requête et le calcul de vecteurs de phrases/documents.

b. Calcul des descripteurs

Après la phase de structuration (pré-indexation), cette sous-section a pour objectif de calculer les descripteurs associés aux différentes unités linguistiques (documents et phrases) et de les stocker de manière cohérente dans les fiches. À ce stade, il ne s'agit pas encore de construire des structures d'index dédiées à la recherche, mais de produire des représentations vectorielles fiables, traçables et réutilisables, qui serviront de base aux étapes ultérieures d'indexation et d'optimisation.

Il ne s'agit donc plus d'implémenter des algorithmes de calcul isolés, mais de mettre en place des fonctions d'orchestration capables de :

- parcourir les fiches correspondant aux unités linguistiques concernées (documents ou phrases),
- récupérer les informations nécessaires (texte, tokens, configuration de prétraitement),
- appeler les fonctions de calcul atomiques étudiées précédemment,
- stocker les vecteurs obtenus ainsi que leurs paramètres de calcul dans les fiches.

Les fonctions de calcul atomique vues en séance 4 pour les descripteurs symboliques, ainsi qu'en section II.2 pour le plongement vectoriels, sont réutilisées telles quelles et ne sont pas réécrites ici.

i. Principe général du calcul des descripteurs

Les descripteurs peuvent être calculés :

- pour plusieurs configurations de prétraitement,
- pour plusieurs langues,
- pour plusieurs corpus (corpus global et sous-corpus),
- pour plusieurs types de descripteurs (symboliques ou vectoriels),
- avec ou sans normalisation.

Pour les documents appartenant au corpus, les descripteurs sont pré-calculés et stockés dans les fiches, afin de permettre une exploitation rapide lors des phases d'indexation et de recherche interactive prévues ultérieurement (séance 7).

Pour les requêtes ou les documents extérieurs au corpus, les calculs pourront être réalisés à la volée, en s'appuyant sur les mêmes fonctions de calcul.

Il n'est pas attendu que toutes les combinaisons possibles de descripteurs, de configurations de prétraitement et de paramètres soient systématiquement calculées. Vous êtes invités à sélectionner un ensemble de représentations cohérent avec vos objectifs d'expérimentation et à justifier vos choix.

ii. Calcul des descripteurs symboliques

Les fonctions définies dans cette section parcourent les fiches concernées, récupèrent les tokens correspondant à une configuration de prétraitement donnée, appellent les fonctions atomiques de calcul des descripteurs symboliques vues en séance 4, puis stockent les vecteurs obtenus dans les fiches.

- Écrire la fonction **calculer_bow_fiches (fiches, vocabulaire, type_bow="binaire", config_id=None, normalisation=None)** qui calcule et stocke les vecteurs Bag-Of-Words pour les fiches sélectionnées.
- Écrire la fonction **calculer_tf_fiches (fiches, vocabulaire, config_id=None, normalisation=None)** qui calcule et stocke les vecteurs TF dans les fiches.
- Écrire la fonction **calculer_tfidf_fiches (fiches, idf_vecteur, config_id=None, normalisation=None)** qui calcule et stocke les vecteurs TF-IDF à partir des vecteurs TF déjà disponibles.
- Écrire la fonction **calculer_bm25_fiches (fiches, vocabulaire, stats_corpus, parametres_bm25, config_id=None, normalisation=None)** qui calcule et stocke les vecteurs BM25 à partir des statistiques du corpus.

Pour chaque descripteur calculé, les paramètres de calcul associés (configuration de prétraitement, type de normalisation, paramètres spécifiques) sont également stockés dans les fiches, afin de garantir la traçabilité et la reproductibilité des représentations.

Les fonctions d'orchestration définies dans cette section ont pour rôle de coordonner le calcul et le stockage des descripteurs, et non de réimplémenter les méthodes de calcul atomiques étudiées en séance 4. Cette séparation vise à favoriser la modularité, la réutilisation du code et l'évolution progressive du moteur de recherche.

iii. Calcul des descripteurs vectoriels (embeddings)

Les descripteurs vectoriels sont calculés selon le même principe d'orchestration que les descripteurs symboliques. Les fonctions définies ci-dessous exploitent les modèles de plongement étudiés en section II.2 et respectent les niveaux de granularité définis précédemment (phrase, document).

- Écrire la fonction **calculer_embeddings_phrases (fiches, modele_embeddings)** qui parcourt les fiches de type phrase, récupère les textes ou tokens nécessaires, calcule les embeddings de phrases à l'aide du modèle fourni et ajoute les vecteurs obtenus et leurs paramètres dans les fiches.

- Écrire la fonction **calculer_embeddings_documents_mots** (**fiches**, **modele_embeddings**) qui parcourt les fiches de type document, calcule les embeddings de documents par agrégation des embeddings de mots et stocke les vecteurs obtenus dans les fiches.
- Écrire la fonction **calculer_embeddings_documents_phrases** (**fiches**, **modele_embeddings**) qui calcule les embeddings de documents par agrégation des embeddings de phrases déjà disponibles dans les fiches et stocke les vecteurs correspondants.
- Écrire la fonction **calculer_embeddings_documents_doc2vec** (**fiches**, **modele_doc2vec**) qui utilise un modèle Doc2Vec pour inférer directement les vecteurs de documents à partir de leur contenu et stocke les embeddings obtenus dans les fiches.

Les vecteurs calculés constituent des représentations prêtes à être indexées, mais ne sont pas encore organisés dans des structures optimisées pour la recherche. Cette organisation sera abordée dans les sous-sections suivantes.

c. Indexation des descripteurs

Après la structuration des descripteurs (pré-indexation) et leur calcul, l'étape d'indexation vise à préparer les données pour une exploitation efficace, sans encore réaliser la recherche proprement dite. Contrairement aux fiches, qui constituent une structure de stockage riche et traçable, les index sont des structures dérivées, construites à partir des fiches, dont l'objectif principal est d'accélérer les accès et les calculs nécessaires à la recherche ultérieure.

L'indexation ne consiste donc ni à recalculer les descripteurs, ni à effectuer le classement des documents. Elle transforme les représentations existantes stockées dans les fiches en structures adaptées aux calculs nécessaires au moteur de recherche. Elle consiste à construire des vues organisées et optimisées des descripteurs afin de :

- accélérer les calculs de similarité ;
- faciliter la comparaison entre requêtes et documents ;
- préparer la recherche interactive (séance 7).

En d'autres termes :

- Les **fiches** servent à *stocker* les informations.
- Les **index** servent à *calculer rapidement* à partir de ces informations.

Dans ce projet, l'indexation est abordée de manière pédagogique, en distinguant deux niveaux complémentaires :

- l'indexation des descripteurs sous forme de vues matricielles,
- l'indexation opérationnelle, qui regroupe des pré-calculs destinés à accélérer les traitements.

Les index sont construits à partir des fiches et peuvent être sauvegardés et rechargés afin d'éviter des reconstructions coûteuses.

i. Indexation symbolique et vectorielle (vues matricielles)

L'indexation symbolique et vectorielle consiste à extraire, à partir des fiches, des vues matricielles qui représentent l'ensemble du corpus sous une forme directement exploitable par les algorithmes de similarité, sans se substituer aux fiches elles-mêmes mais comme des représentations dérivées destinées à faciliter les calculs numériques.

Pour les descripteurs symboliques (BOW, TF, TF-IDF, BM25) :

- on construit une **matrice document-terme** ou **phrase-terme** ;
- chaque ligne correspond à un document ou une phrase ;
- chaque colonne correspond à un terme du vocabulaire ;
- le nombre de colonnes est égal à la taille du vocabulaire.

Pour les descripteurs vectoriels (embeddings) :

- on construit une **matrice dense de vecteurs** ;
- chaque ligne correspond à un document ou une phrase ;
- chaque colonne correspond à une dimension latente du modèle ;
- le nombre de colonnes est égal à la dimension du vecteur d'embedding (par exemple 100, 300, etc.).

Dans les deux cas, la matrice est accompagnée de structures auxiliaires permettant de relier chaque ligne de la matrice à la fiche correspondante et d'associer efficacement les opérations matricielles aux unités linguistiques présentes dans ces fiches. L'ensemble de ces éléments est regroupé au sein d'une structure d'index, qui constitue l'unité organisationnelle centrale du dispositif. Chaque index symbolique ou vectoriel comprend typiquement :

- une matrice de descripteurs (symbolique ou dense),
 - un mapping identifiant → ligne, qui est une table de correspondance indiquant, pour chaque document ou phrase du corpus, la position de son vecteur dans la matrice de descripteurs. Il permet de relier efficacement les représentations matricielles aux fiches correspondantes.
 - éventuellement un vocabulaire, des métadonnées associées, la configuration de prétraitement, le type de descripteur, des paramètres de calcul, type de modèle, etc.
-
- Écrire la fonction **construire_index_symbolique(fiches, type_descripteur, config_id=None)** qui parcourt les fiches du corpus, extrait les vecteurs symboliques correspondant au type de descripteur et à la configuration choisie, construit une matrice document-terme (ou phrase-terme), crée un mapping identifiant → ligne de matrice, et retourne une structure d'index symbolique cohérente regroupant l'ensemble de ces éléments.
 - Écrire la fonction **construire_index_embeddings(fiches, type_embeddings, config_id=None)** qui extrait les embeddings stockés dans les fiches, construit une matrice dense de vecteurs, crée un mapping identifiant → ligne, et retourne une structure d'index vectoriel prête à être utilisée pour des calculs de similarité.

L'index, qu'il soit symbolique ou vectoriel, permet d'effectuer efficacement des recherches lexicales et sémantiques, de calculer des similarités (par exemple cosinus)

entre unités linguistiques, de classer les documents, et de soutenir l'expansion de requête à partir des plongements. Dans certaines situations, le corpus peut évoluer après la construction initiale des index : ajout d'un nouveau document, rechargement partiel de fiches, ou intégration de documents externes. Lorsque les descripteurs correspondants sont déjà calculés et stockés dans les fiches, il est possible de mettre à jour les index existants sans les reconstruire entièrement. Les fonctions suivantes illustrent ce principe de mise à jour incrémentale des index, limité ici à des opérations simples et pédagogiques.

- Écrire la fonction **mettre_a_jour_index_embeddings(index_embeddings, fiche)** qui ajoute dynamiquement les embeddings d'une nouvelle unité linguistique à un index vectoriel existant. Cette fonction suppose que la fiche fournie contient déjà les embeddings calculés. Elle met à jour la matrice des vecteurs, le mapping identifiant $\rightarrow$ ligne et les métadonnées associées, sans reconstruire l'ensemble de l'index. Elle ne réalise aucun calcul d'embeddings et ne modifie pas les descripteurs, mais se limite à l'actualisation de la structure d'index.
- Écrire la fonction **mettre_a_jour_index_symbolique(index_symbolique, fiche)** qui ajoute les descripteurs symboliques d'une nouvelle unité linguistique à un index symbolique existant, en mettant à jour la matrice document-terme et les structures associées.

ii. Indexation opérationnelle

Une fois les index symboliques et vectoriels construits sous forme de vues matricielles, certaines opérations peuvent être pré-calculées ou certaines structures dérivées peuvent être construites afin d'accélérer les traitements ultérieurs. Cette étape, appelée indexation opérationnelle, consiste à enrichir les index existants par des informations supplémentaires destinées à réduire le coût des calculs répétitifs lors de la recherche interactive, sans modifier les descripteurs eux-mêmes et sans effectuer le calcul des scores. L'indexation opérationnelle ne crée donc pas de nouvelles représentations textuelles ou vectorielles. Elle repose sur les descripteurs déjà calculés et stockés dans les index, et vise à réorganiser ou compléter ces informations afin de rendre les opérations futures plus efficaces. Elle constitue une étape intermédiaire essentielle entre la construction des index et leur exploitation lors de la recherche.

Parmi les opérations possibles, on distingue notamment :

- le pré-calcul de quantités numériques utilisées de manière répétée lors des calculs de similarité ;
- la construction de structures d'accès rapide permettant de limiter le nombre d'opérations nécessaires lors du calcul des scores.

Ces optimisations sont particulièrement importantes dans un contexte interactif, où une même base de documents est interrogée à plusieurs reprises avec des requêtes différentes.

Dans le cadre de ce projet, l'indexation opérationnelle reste volontairement simple et pédagogique. Elle se limite à des mécanismes fondamentaux de la recherche d'information, sans introduire d'algorithmes avancés de recherche approximative ni de techniques d'optimisation complexes.

1) Pré-calculs liés aux mesures de similarité vectorielles

Pour toute représentation vectorielle, qu'elle soit issue de descripteurs symboliques pondérés (TF, TF-IDF, BM25) ou d'embeddings denses, le calcul de la similarité cosinus entre une requête et un ensemble de documents nécessite le calcul répété des normes des vecteurs. Afin d'éviter de recalculer ces normes à chaque requête, celles-ci peuvent être pré-calculées une seule fois pour les documents du corpus et stockées directement dans l'index opérationnel associé. Ces optimisations ne dépendent pas de la "famille" du descripteur, mais des opérations mathématiques que l'on répète lors du calcul de similarité. La norme de la requête, quant à elle, est calculée à la volée, car la requête est dynamique, contrairement aux documents du corpus.

Ce pré-calcul permet de :

- réduire le nombre d'opérations arithmétiques lors du calcul de similarité ;
 - rendre le calcul des scores plus rapide et plus stable numériquement ;
 - simplifier l'implémentation de la similarité cosinus lors de la recherche.
-
- Écrire la fonction **precalculer_normes_embeddings(index_embeddings)** qui calcule, pour chaque vecteur d'embeddings associé aux documents ou aux phrases du corpus, la norme du vecteur et la stocke dans une structure dérivée associée à l'index d'embeddings. Cette structure permet d'accélérer les calculs de similarité cosinus lors de la recherche, en évitant le recalcul répété des normes des vecteurs du corpus.
 - Écrire la fonction **precalculer_normes_symboliques(index_symbolique)** qui calcule et stocke les normes des vecteurs symboliques associés aux documents ou aux phrases du corpus (BOW, TF, TF-IDF ou BM25). Ces normes sont utilisées pour accélérer le calcul de la similarité cosinus entre une requête et les documents indexés, sans modifier les descripteurs symboliques eux-mêmes.

2) Pré-calculs spécifiques aux modèles symboliques

Pour les descripteurs symboliques, qu'ils soient binaires (BOW) ou pondérés (BOW_occ, TF, TF-IDF, BM25), certaines opérations peuvent être optimisées en construisant des structures dérivées destinées à réduire le nombre de calculs nécessaires lors de la recherche. Une optimisation fondamentale consiste à limiter le nombre de documents à évaluer pour une requête donnée. En effet, pour une requête contenant un nombre restreint de termes, il est inutile de comparer la requête avec l'ensemble des documents du corpus : seuls les documents contenant au moins un des termes de la requête sont susceptibles de produire un score non nul. Cette logique est mise en œuvre à travers un index inversé, une structure qui associe chaque terme du vocabulaire à la liste des documents dans lesquels il apparaît. Pour les modèles pondérés (TF-IDF, BM25), cet index peut également stocker les pondérations partielles des termes dans chaque document.

Exemple :

Soit un corpus de 4 documents :	Index Inversé associé :																						
<table><tr><th>DocID</th><th>Texte</th></tr><tr><td>D1</td><td>"chat noir"</td></tr><tr><td>D2</td><td>"chien blanc"</td></tr><tr><td>D3</td><td>"chat blanc"</td></tr><tr><td>D4</td><td>"oiseau noir"</td></tr></table>	DocID	Texte	D1	"chat noir"	D2	"chien blanc"	D3	"chat blanc"	D4	"oiseau noir"	<table><tr><th>Terme</th><th>Documents</th></tr><tr><td>chat</td><td>D1, D3</td></tr><tr><td>noir</td><td>D1, D4</td></tr><tr><td>blanc</td><td>D2, D3</td></tr><tr><td>chien</td><td>D2</td></tr><tr><td>oiseau</td><td>D4</td></tr></table>	Terme	Documents	chat	D1, D3	noir	D1, D4	blanc	D2, D3	chien	D2	oiseau	D4
DocID	Texte																						
D1	"chat noir"																						
D2	"chien blanc"																						
D3	"chat blanc"																						
D4	"oiseau noir"																						
Terme	Documents																						
chat	D1, D3																						
noir	D1, D4																						
blanc	D2, D3																						
chien	D2																						
oiseau	D4																						

Pour une requête contenant « chat blanc », seuls D1, D2 et D3 seront pris en compte lors du calcul des scores. Les autres documents sont écartés avant toute opération de similarité, ce qui réduit significativement le nombre de calculs nécessaires.

- Écrire la fonction **construire_index_inverse(fiches, type_descripteur, config_id=None)** qui construit un index inversé à partir des fiches du corpus, en associant à chaque terme du vocabulaire la liste des identifiants des documents (ou phrases) dans lesquels il apparaît. Pour les descripteurs symboliques pondérés, l'index inversé peut également stocker les pondérations associées à chaque terme dans chaque document, afin de limiter les calculs ultérieurs aux seules unités linguistiques pertinentes lors de la recherche.
- Écrire la fonction **mettre_a_jour_index_inverse(index_inverse, fiche)** qui met à jour dynamiquement un index inversé existant à partir d'une fiche nouvellement ajoutée ou enrichie. Cette fonction ajoute les termes présents dans la fiche et leurs associations aux documents correspondants, sans reconstruire l'intégralité de l'index inversé, et garantit la cohérence de la structure pour les recherches futures.

Ces étapes finalisent la préparation des données pour la recherche interactive. Le calcul effectif des similarités, le classement des résultats et leur exploitation dans une interface utilisateur seront abordés dans la séance dédiée à l'Interface Homme-Machine.

d. Persistance

Après le calcul et la structuration des descripteurs, il est nécessaire de pouvoir sauvegarder sur disque afin de réutiliser les vecteurs sans recalculer à chaque expérimentation, de faciliter le partage ou le transfert des descripteurs entre différentes sessions ou machines, et de préparer le moteur de recherche à une exploitation efficace des données. La persistance consiste à stocker sur disque à la fois les fiches représentant les unités linguistiques (phrases ou documents) avec leurs descripteurs et métadonnées, et les index ou matrices dérivés de ces fiches, afin de pouvoir les recharger ultérieurement sans perte d'information ni recalcul. La structure mémoire est conçue

pour un accès rapide lors du calcul des similarités ou de l'expansion de requête, et la persistance garantit cohérence, lisibilité et réutilisation des données à travers différentes sessions ou environnements.

Le format et l'organisation du stockage persistant ne sont pas imposés. Vous pouvez choisir de sauvegarder les fiches et les index dans un fichier unique ou dans plusieurs fichiers, par corpus, configuration de prétraitement ou type de descripteur. Des formats standards de sérialisation, tels que JSON (pour les métadonnées), Pickle (pour des structures Python complexes), ou des formats binaires dédiés aux tableaux numériques (par exemple ceux proposés par NumPy ou HDF5), peuvent être utilisés pour sauvegarder les fiches et les descripteurs. Les exemples de formats cités ont uniquement pour objectif de fournir des points d'entrée pour la recherche de documentation ; aucun format spécifique n'est imposé. Le choix du format dépend des contraintes de taille, de lisibilité et de rapidité de chargement. Il n'est pas attendu que vous maîtrisiez ces formats au préalable. L'objectif est d'explorer un mécanisme simple de sauvegarde et de rechargement des descripteurs, en s'appuyant sur la documentation disponible.

- Écrire la fonction **sauvegarder_fiches(fichier, dictionnaire_global)** qui sauvegarde l'ensemble des fiches (documents et phrases) sur disque, en conservant pour chaque descripteur les vecteurs et les paramètres ayant conduit à leur calcul.
- Écrire la fonction **sauvegarder_fiche_unitaire(fichier, fiche)** qui sauvegarde sur disque la fiche correspondant à une unité linguistique (phrase ou document), uniquement si le statut de ses descripteurs est marqué comme « *calculé* ». Cette fonction permet de persister une fiche nouvellement enrichie sans réécrire l'ensemble des données déjà stockées.
- Écrire la fonction **sauvegarder_fiches_calculees(dico_global, dossier_sortie)** qui parcourt l'ensemble des fiches du dictionnaire global et appelle la fonction *sauvegarder_fiche_unitaire* pour chaque fiche dont au moins un descripteur a le statut « *calculé* ». Cette approche permet de sauvegarder les fiches individuellement, par exemple dans un fichier par fiche, selon votre choix d'organisation.
- Écrire la fonction **sauvegarder_index_symbolique(fichier, index_symbolique)** qui parcourt la structure d'index symbolique et sauvegarde sur disque tous les éléments nécessaires pour sa réutilisation ultérieure : la matrice document-terme ou phrase-terme, le vocabulaire associé, le mapping identifiant → ligne, ainsi que les métadonnées. L'objectif est de permettre de recharger cet index sans recalculer les vecteurs, tout en conservant la cohérence entre les documents (ou phrases) et leurs représentations symboliques.
- Écrire la fonction **sauvegarder_index_embeddings(fichier, index_embeddings)** qui sauvegarde sur disque l'ensemble des informations contenues dans un index vectoriel : la matrice dense des embeddings, le mapping identifiant → ligne, et toutes les métadonnées nécessaires pour effectuer des calculs rapides de similarité. Elle permet de recharger l'index vectoriel ultérieurement, sans recalculer les vecteurs, et de préparer le moteur de recherche pour la recherche interactive ou l'expansion de requête.
- Écrire la fonction **sauvegarder_normes_embeddings(fichier, index_embeddings)** qui sauvegarde sur disque la structure contenant les normes des vecteurs d'embeddings pré-calculées pour tous les documents ou phrases du

corpus. Cette sauvegarde permet de recharger rapidement ces informations lors des calculs de similarité cosinus sans recalcul.

- Écrire la fonction **sauvegarder_normes_symboliques(fichier, index_symbolique)** qui sauvegarde sur disque les normes pré-calculées des vecteurs symboliques (BOW, TF, TF-IDF, BM25) associées aux documents ou phrases du corpus. Cette persistance permet d'accélérer les futures recherches en réutilisant les normes déjà calculées.
- Écrire la fonction **sauvegarder_index_inverse(fichier, index_inverse)** qui sauvegarde sur disque la structure d'index inversé construite pour les descripteurs symboliques. L'index sauvegardé comprend pour chaque terme du vocabulaire la liste des documents ou phrases correspondants, ainsi que les pondérations éventuelles pour les modèles pondérés, afin de limiter les recalculs lors des requêtes ultérieures.
- Écrire la fonction **mettre_a_jour_index_inverse_persistant(fichier, index_inverse)** qui met à jour la version sauvegardée sur disque d'un index inversé existant avec de nouvelles fiches ou documents ajoutés, en intégrant les nouveaux termes et associations sans réécrire entièrement l'index. Cette fonction assure la cohérence entre l'index en mémoire et sa version persistante.

e. Rechargement

Après la persistance des descripteurs sur disque, il est nécessaire de pouvoir les recharger en mémoire afin de réutiliser un moteur de recherche existant sans recalculer l'ensemble des représentations. Le rechargement constitue l'étape complémentaire de la persistance : il permet de reconstruire en mémoire les structures de données nécessaires à la recherche, à l'indexation et à l'expérimentation interactive. Cette étape concerne à la fois les fiches associées aux unités linguistiques (documents ou phrases) et les index dérivés (symboliques, vectoriels et opérationnels). Le rechargement doit garantir que les fiches et les index reconstruits restent immédiatement exploitables par les fonctions de calcul de similarité, d'indexation ou d'interface Homme-Machine, sans distinction entre des descripteurs calculés lors de la session courante ou chargés depuis le disque. Il doit également permettre l'intégration progressive de nouvelles fiches ou de nouveaux descripteurs dans les structures existantes, en cas de rechargement partiel.

Les objectifs principaux du rechargement sont les suivants :

- reconstruire le dictionnaire global de fiches à partir des données sauvegardées,
 - restaurer les descripteurs et leurs paramètres associés,
 - préserver les liens entre les fiches, les représentations vectorielles et les structures dérivées (matrices, index),
 - éviter tout recalcul inutile des descripteurs déjà disponibles.
 - maintenir la cohérence des identifiants, des configurations de prétraitement et des métadonnées.
-
- Écrire la fonction **charger_fiches(fichier)** qui charge depuis le disque l'ensemble des fiches sauvegardées et reconstruit le dictionnaire global des unités linguistiques (documents et phrases), avec leurs descripteurs, paramètres et métadonnées.

- Écrire la fonction **charger_fiche_unitaire(fichier)** qui charge depuis le disque la fiche correspondant à une unité linguistique donnée et la retourne sous forme exploitable en mémoire.
- Écrire la fonction **charger_fiches_depuis_dossier(dossier)** qui parcourt un ensemble de fichiers de fiches sauvegardées individuellement et reconstruit le dictionnaire global des fiches à partir de ces fichiers.
- Écrire la fonction **integrer_fiche_chargee(dico_global, fiche)** qui ajoute une fiche chargée dans le dictionnaire global en vérifiant la cohérence des identifiants, des configurations de prétraitement et des descripteurs déjà présents.
- Écrire la fonction **charger_index_symbolique(fichier)** qui recharge un index symbolique depuis le disque, en reconstruisant la matrice document-terme ou phrase-terme, le vocabulaire associé, le mapping identifiant → ligne, ainsi que les métadonnées.
- Écrire la fonction **charger_index_embeddings(fichier)** qui recharge un index vectoriel depuis le disque, en reconstruisant la matrice dense des embeddings, le mapping identifiant → ligne et toutes les métadonnées nécessaires pour le calcul rapide de similarité.
- Écrire la fonction **verifier_coherence(dictionnaire_global)** qui contrôle l'unicité des identifiants et la compatibilité des métadonnées avec les configurations attendues et la cohérence globale des fiches.
- Écrire la fonction **gestion_vecteurs_absents(fiche, strategie='ignore')** qui gère les mots, phrases ou documents dont le vecteur n'est pas présent, selon la stratégie choisie (**ignore**, **alerte**, **recalcul**), elle permet d'éviter des erreurs lors des calculs de similarité.
- Écrire la fonction **charger_normes_embeddings(fichier)** qui recharge depuis le disque la structure contenant les normes pré-calculées des vecteurs d'embeddings pour tous les documents ou phrases du corpus. Cette opération permet de restaurer rapidement les informations nécessaires au calcul des similarités cosinus sans recalcul.
- Écrire la fonction **charger_normes_symboliques(fichier)** qui recharge depuis le disque les normes pré-calculées des vecteurs symboliques (BOW, TF, TF-IDF, BM25) pour tous les documents ou phrases. La réutilisation de ces normes accélère les recherches et assure la cohérence avec l'index symbolique principal.
- Écrire la fonction **charger_index_inverse(fichier)** qui recharge depuis le disque la structure d'index inversé construite pour les descripteurs symboliques. Cette fonction permet de restaurer la liste des documents ou phrases associés à chaque terme du vocabulaire, ainsi que les pondérations partielles pour les modèles TF-IDF ou BM25.
- Écrire la fonction **charger_index_inverse_mise_a_jour(fichier)** qui recharge et fusionne les mises à jour d'un index inversé existant depuis le disque. Elle permet d'intégrer de nouvelles fiches ou documents tout en conservant la cohérence avec l'index inversé déjà en mémoire, sans reconstruire entièrement la structure.

f. Transition vers la séance Interface Homme-Machine

À l'issue des étapes de structuration, de calcul, d'indexation et de persistance, le moteur de recherche dispose désormais de toutes les structures nécessaires à son exploitation. Les fiches contenant les unités linguistiques et leurs descripteurs, ainsi que les index

symboliques et vectoriels et leurs éventuels enrichissements opérationnels, constituent une base cohérente et prête à l'emploi.

Cette section s'est volontairement limitée à la préparation des données et des structures, sans aborder directement la recherche interactive ni le calcul des scores. Ces aspects relèvent de la séance suivante, consacrée à l'Interface Homme-Machine. Vous y utiliserez les fiches, les index et les pré-calculs mis en place ici pour :

- formuler des requêtes utilisateur ;
- calculer des similarités entre requêtes et documents ou phrases;
- classer et afficher les résultats ;
- explorer et comparer différentes stratégies de recherche (symbolique, vectorielle ou hybride).

Cette séparation permet de distinguer clairement :

- la préparation du moteur de recherche (gestion des descripteurs et indexation),
- de son exploitation interactive au sein d'une interface utilisateur.

Les structures construites dans cette section constituent ainsi le socle technique sur lequel reposera l'ensemble des interactions étudiées dans la séance Interface Homme-Machine.